

NATIONAL UNIVERSITY OF SCIENCES AND
TECHNOLOGY ISLAMABAD

MASTER THESIS

**Discriminative Splitting of LSTM-RNN
Deep Neural Networks for Robust Speech
Recognition**

Author:
Muhammad Irfan

Supervisor:
Dr. Ali Tahir

*A thesis submitted in fulfillment of the requirements
for the degree of Master of Computer Science
in the*

School of Electrical Engineering and Computer Science

May 4, 2018

Declaration of Authorship

I, Muhammad IRFAN, declare that my thesis titled, “Discriminative Splitting of LSTM-RNN Deep Neural Networks for Robust Speech Recognition” and work carried out is my own. I confirm that:

- This work was done during my research degree at this University.
- Published work related to my research states properly.
- Any quoted text from book is stated if any.
- I have acknowledged those who helped me during the research.
- Thesis work is done by myself and those who jointly work with me has been mentioned.

Signed:

Date:

“My thirst led me to do things which people think are impossible.”

Muhammad Irfan

NATIONAL UNIVERSITY OF SCIENCES AND TECHNOLOGY ISLAMABAD

Abstract

School of Electrical Engineering and Computer Science

Master of Computer Science

Discriminative Splitting of LSTM-RNN Deep Neural Networks for Robust Speech Recognition

by Muhammad IRFAN

DNN and RNN are becoming popular to for solving complex problems such as speech recognition, machine translation, natural language processing and image recognition. One major aspect of RNNs are their ability to learn though propagation using weights. Weights are important for reducing the errors and for optimization. Normally weights are initialized using randomization and are adjusted during training. We present parameter splitting concept for training the DNN and RNN for problems based on sequences such as speech recognition. We start from a small model and gradually increase the number of parameters of RNN by splitting the weights and appending together. Random noise is added to boost the weights. Our splitting technique generates better word error rate and reduced overtraining of the model...

Acknowledgements

I would like to thank my advisor, *Dr. Muhammad Ali Tahir* for guiding me through research phase. I also want to thank committee members (*Dr. Faisal Shafait, Dr. Hassan Aqeel Khan, Dr. Ahmad Salman*) for their guidance and helping me to think out of the box, create new ideas and implement them...

Contents

Declaration of Authorship	iii
Abstract	vii
Acknowledgements	ix
1 Introduction & Related Work	1
1.1 Introduction	1
1.2 Related Work	2
2 A Short Intro to Automatic Speech Recognition	3
2.1 Introduction	3
2.1.1 Speech Recognition System	3
2.1.2 ASR Toolkits	4
2.2 Applications	4
2.2.1 Navigation	5
2.2.2 Dictation	5
2.2.3 Home Automation	5
2.2.4 Customer Support	6
3 Discriminative Parameter Splitting	7
3.1 The Concept	7
3.2 Pictorial Representation of Splitting	7
3.3 Limit of Splitting	9
4 Recurrent Neural Networks	11
4.1 Peephole LSTM-RNN	11
4.1.1 Memory Cell	11
4.1.2 Equations	12
4.2 Architecture of Peephole LSTM	14
5 Implementation	17
5.1 Dataset and Toolkit	17
5.1.1 Dataset	17
5.1.2 Toolkit	17
5.2 Procedure to Train Proposed Model	17
5.3 Build Initial and Final Model	17
5.3.1 MFCC Feature Extraction and Alignment Model	18
5.3.2 Initialization of Hidden Layer Weight Matrices	18
5.3.3 Creating Initial and Final Acoustic Models	18
5.4 Training and Model Generation	19
5.4.1 Updation of Initial Acoustic Model	19
5.4.2 Updation of Final Acoustic Model	19
5.4.3 Algorithm	20

5.5	Model Configurations and Modifications of Parameters	21
5.5.1	Configurations of LSTM Layers	21
5.5.2	Columns Extension of Matrix	21
5.5.3	Rows Extension of Matrix	24
5.5.4	Random Noise	24
6	Results	27
6.0.1	Cost	28
6.0.2	WER	28
7	Conclusion & Future Work	31
7.1	Conclusion	31
7.2	Future Work	31
A	Frequently Asked Questions	33
A.1	Which toolkit and data set is used for experiments?	33
A.2	Is splitting a general model or can be used only in Peephole LSTM? . .	33
	Bibliography	35

List of Figures

2.1	General Speech Recognition System	3
2.2	Navigation using speech	5
2.3	Dictation: Speech to text	5
2.4	Customer Support	6
3.1	Parameter Splitting by extending the nodes of hidden layer. $w_{ij} \in W_{ij}$ is weight and $b_i \in b$ is bias value of their respective gates.	8
3.2	Extension columns of weight matrix W_o of output layer by 1 time.	8
3.3	Extension rows of weight matrix W_o of output layer by 1 time.	9
4.1	LSTM-RNN memory cell (Sak, Senior, and Beaufays, 2014). Doted lines with arrows connecting the cell state to input, output and forget gate are peephole connections. g is a sigmoid function and h is a $\tanh$ function. c_t is cell state and r_t is recurrent connection. i_t , o_t and f_t are input, output and forget gate of hidden layer.	12
4.2	Input i_t , output o_t and forget f_t gate in Peephole LSTM-RNN. <i>SUM</i> and <i>PRODUCT</i> are two matrix operations.	14
4.3	Input i_t , output o_t and forget f_t gate in Peephole LSTM-RNN with given dimensions of gate matrices and bias vectors with 512 dimensions.	14
4.4	Input i_t , output o_t and forget f_t gate in Peephole LSTM-RNN with given dimensions of gate matrices and bias vectors with 2048 dimensions	15
5.1	Algorithm for our splitting parameter model	20
5.2	Configurations of LSTM with 1024 matrix dimension	22
5.3	Configurations of LSTM with 2048 matrix dimension	23
5.4	Extension of columns of a matrix X , c represents the change in columns and r is random noise by adding R matrix.	23
5.5	Extension of rows of a matrix X , r represents the change in rows and r is random noise by adding R matrix.	24
6.1	Loss function comparison of three layer LSTM.	28
6.2	Loss function comparison of five layer LSTM.	29
6.3	Effects on WER on Non-discriminative vs discriminative for three Layer (a)and five layer (b)	29

List of Tables

6.1 Experiments on three, five and seven layer LSTM	27
---	----

List of Abbreviations

AI	Artificial Intelligence
ASR	Automatic Speech Recognition
BLSTM	Bidirectional Long Short Term Memory
DNN	Deep Neural Network
FMLLT	Feature Space Maximum Likelihood Linear Transform
FMMI	Feature Space Maximum Mutual Information
GMM	Gaussian Mixture Model
GRU	Gated Recurrent Unit
G2P	Grapheme-to- Phoneme
HMM	Hidden Markov Model
LDA	Linear Discriminative Analysis
MFCC	Mel Frequency Cepstral Coefficient
MMI	Maximum Mutual Information
LSTM	Long Short Term Memory
ML	Machine Learning
MLLT	Maximum Likelihood Linear Transform
RNN	Recurrent Neural Network
SAT	Speaker Adaptive Training
WER	Word Error Rate

List of Symbols

μ	Mean
σ	Standard Deviation
U	Uniform Distribution
N	Normal Distribution
ϕ	Softmax
h	Tanh
C	Constant (0.0001)

*This thesis is dedicated to my parents and teachers for their
love and guidance...*

Chapter 1

Introduction & Related Work

1.1 Introduction

Recurrent neural networks have become state-of-the-art models to solve challenging problems such as natural language processing, machine translation, speech recognition, image processing. To solve such complex problems new algorithms are being proposed.

A decade ago these problems were being solved by classic algorithms due to limitation of hardware and also software. Due to advancement in technology, hardware has been improved to solve such problems. GPUs are being used to train DNN/RNN models within hours and provide better accuracy.

Speech recognition is a complex problem. It requires signal processing techniques, language modeling and lexicon generation which then can be used with HMMGMM or DNNRNN models to build a speech recognition system. Currently DNN and RNN are being used for research and development purpose. (Sundermeyer et al., 2013) shows that RNN have overtaken HMM/GMM based models.

Deep and recurrent neural networks consists of cycles that can be used to solve the sequence problems such as speech recognition and machine translation. DNN models are good as compared to HMM/GMM models but DNN has better accuracy. One major drawback of HMM/GMM models is that they can't predict better within a time frame or window. On the other hand DNN has the ability to remember information for a longer time. At some point DNN models can stuck vanishing gradient point due sticking on in local minimum and remains there. Luckily RNN models has solved the issue using backpropagation through time.

RNN has the ability to remember information and avoid gradient vanishing problems. It maps better the sequence problems. It uses back and forward propagation through time to reduce the errors. Weights play important role in forward and back propagation to minimize the errors and help to get good word error rate.

Weight initialization techniques (Murru and Rossini, 2016; Qiao, Li, and Li, 2016) has been studied and available in literature. It was applied to feed forward and deep neural network and has good effects on WER. Bayesian (Qiao, Li, and Li, 2016) approach is used with mutually dependent random variables for weight initialization. In (Qiao, Li, and Li, 2016) model, weights are initialized using the input which is provided to feed forward neural network. Weight are initialized using bounds of inputs based on initial and final input values. Binary weight initialization is also experiment in (courbariaux201). It works like a regularizer for propagation. There is also a weight normalization model available (Salimans and Kingma, 2016) using Euclidian norm. But the concept of splitting of parameters is not known to us for RNN models.

Speech recognition (DNN/RNN) cost and word error rate are influenced by weights (Murru and Rossini, 2016; Qiao, Li, and Li, 2016). Our model has attraction

for weights, due to influence on word error rate and cost. Discriminative training with weights can improve the results of a speech recognition system build with RNN model. Our model adds a boost value to the weights to improve the word error rate and reduce the cost during training.

1.2 Related Work

There is a paper on discriminative splitting for GMM models (Tahir, Schlüter, and Ney, 2011). The author describes the effect of splitting on HMM model by splitting the GMM for HMM states. This method shows the good effect on acoustic model. We have used this technique for Peephole LSTM on large corpus to see the effects of splitting of weights of hidden layers.

The concept of adding GMM (Gers and Schmidhuber, 2000) noise to RNN is implemented to simplify generalize the weights.

Gradient noise addition is implemented for improving the deep neural networks in (Neelakantan et al., 2015) which produces the good results for very large neural networks.

We have introduced the concept of splitting the RNN model from small network to large network and added the calculated noise for improvement of WER and cost of training.

Chapter 2

A Short Intro to Automatic Speech Recognition

2.1 Introduction

Automatic speech recognition is a complex task for converting the speech to text (STT). Audio signals of speech cannot be used directly as these signals are in analog form. For computers to understand, these signals are converted into a digital form which is understandable by computers.

Speech recognition was not so popular and was not being used widely in the past. It had very limited use such as dictation and accuracy was not so good. For the past decade, speech recognition has become popular due to advancement in artificial intelligence and machine learning. Accuracy has improved due to advanced algorithms based on neural networks such as deep neural networks (DNN) and recurrent neural networks (RNN).

2.1.1 Speech Recognition System

Every speech recognition system uses some basic models for recognition purpose. First audio is captured with 16-bit sample rate as well as 16kHz frequency. Then this audio is converted to feature vector using signal processing methods such as mel frequency cepstral coefficient (MFCC) and perceptual linear prediction (PLP). A hybrid model can be used by using the advantages of HMM/GMM model and

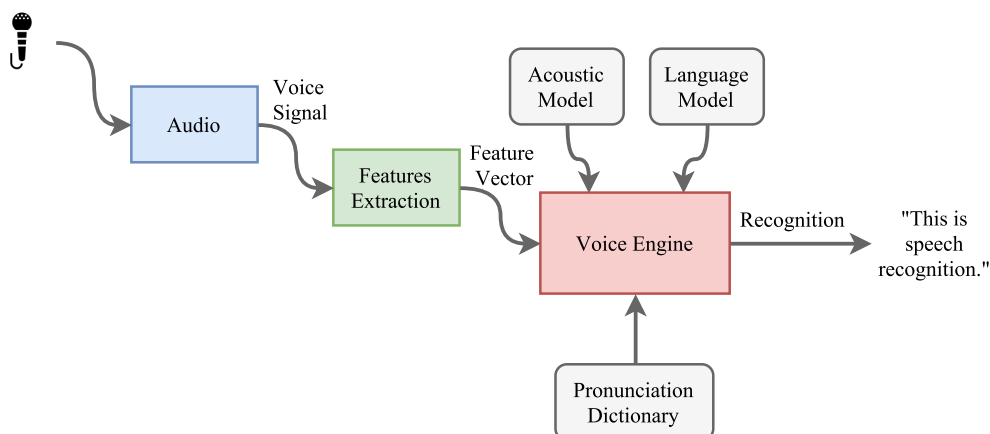


FIGURE 2.1: General Speech Recognition System

DNN/RNN model. These features are used with the combination of acoustic model, language model and pronunciation dictionary. Acoustic model can be built using

classic HMM/GMM model or DNN/RNN model. [Figure 2.1](#) shows a general speech recognition system.

Components of speech recognition system:

- Audio files which are used as input for training a speech recognition system. Normally audios are in 16-bit, 16-khz sample rate files.
- Acoustic model is build using different algorithms such as HMM/GMM or DNN/RNN. Accuracy of speech recognition is based on this model. It maps spoken words to text.
- Pronunciation dictionary contains the lexicon of words. Words are added to dictionary as pronunciation format. For example, word SPEECH lexicon is S P IY1 CH . It provides the mapping between language model and acoustic model.
- Language model consist of word with their probabilities. The most commonly used model is ngram (Brown et al., 1992) model. It helps the voice engine to find out the best word for given phoneme.
- Voice engine is where all the computation is taken place to map the given speech and text. It searches the word for speech using acoustic model and lexicon and produces the output with best matches.

2.1.2 ASR Toolkits

There are many toolkits for speech recognition available as open source or licensed. Each toolkit has it's advantages and disadvantages. It depends on what you want to achieve and build using a toolkit of your necessity.

Some toolkits are used widely which have all the necessary algorithms and features required for research and development of acoustic model. Here are some famous toolkits:

- Kaldi: It is an open source toolkit released in 2011. It has become popular due to DNN and RNN algorithm implementation. It's easy to use and has build in recipes (code) for most commonly used speech corpora i.e. WSJ, LibriSpeech, Voxforge etc.
- CMUSphinx: HMM based speech recognition toolkit. It supports Java, Python and C programming language.
- HTK: It is a HMM based toolkit and also provides neural net algorithms. It is written in C. It can be used for pattern recognition problems.
- RWTH ASR: It is a non-commercial toolkit for speech recognition. It only provides HMM/GMM and feed forward neural networks for research. RNN is not available in this toolkit.

2.2 Applications

Now a days smart phone has ability to understand and execute commands and perform various tasks using speech. Smart phones based on Android and iOS has build in apps such as Google voice and Siri which can understand your speech and convert it to text or other form such as running or closing the apps by just speaking to the phone. Here are some applications of speech recognition:

2.2.1 Navigation

Navigation through maps in cars has become very popular and important applications. Normally a voice assistant is used to guide the drivers to their destination.

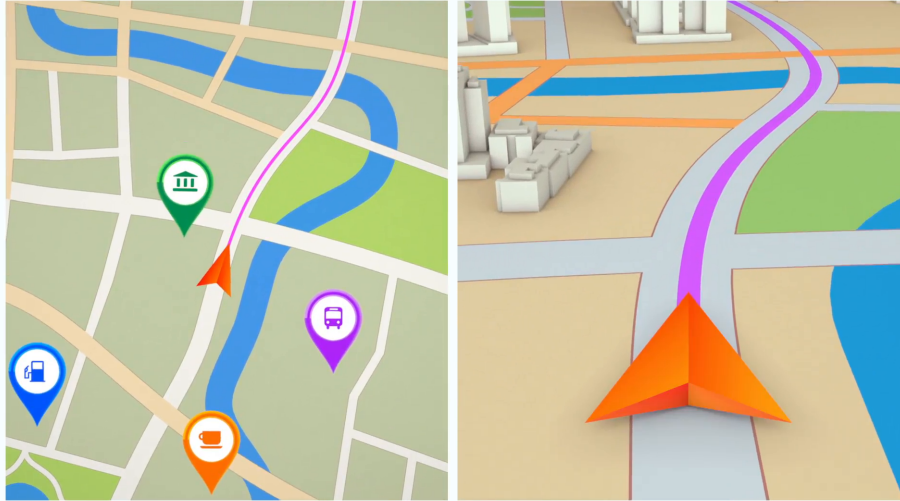


FIGURE 2.2: Navigation using speech

2.2.2 Dictation

Dictation is mostly used as a speech recognition application. It is used normally to write documents such as filling the medical forms, during judiciary talks or write your documents etc.



FIGURE 2.3: Dictation: Speech to text

2.2.3 Home Automation

Voice assistant can be built to automate the home task such as switching on and off the tv, making audio and video call, controlling gates and door of your home etc.

2.2.4 Customer Support

It can be used in commercial application such as getting feedback and activating different services being used in commercial voices services (Mobilink, Telenor, Warid etc)



FIGURE 2.4: Customer Support

Chapter 3

Discriminative Parameter Splitting

3.1 The Concept

RNNs neural networks have hidden layers which take input from previous layer, process it and pass it to next hidden layer. During this process RNN uses weights to filter the input and produces error. Weights are passed back to hidden layers to minimize the error generated in previous step. Bias vectors play important rule for learning the weights and provides help for prediction of the data.

Discriminative parameter splitting method is a technique where parameters of model in DNN, RNN are updated. Weights are generated using randomness and varies between 0 and 1. These weights are important and can extend or reduce error. Weights are improved by adding discriminative noise to weights of LSTM-RNNs gates, cell gates and bias vectors. Parameters of cell gates and biases are modified to increase or decrease the dimensions of matrices and has positive or negative effect on word error rate. This change in weights is propagated in neural network to help in reducing the error. Reducing the size of weights matrices decrease the WER and adding more weights increases WER.

3.2 Pictorial Representation of Splitting

Graphically it is shown in [Figure 3.1](#). Circles are the cells of hidden layer. Weights and bias values are shown as a cube for each cell in hidden layer. Splitting extend the hidden layer dimensions to 1 time. For example if hidden layer consists of 1024 cells then splitting increase the dimension to 2048 and so on depending on how much you want to split. Weight and bias vectors are also extended from 1024 to 2048 as well as cell matrices.

Extension of output matrix is shown in [Figure 3.2](#) for understanding on how we are modifying the weights of hidden layer cells. We are extending the matrix by columns 1 time of output gate. Same process is applied to input and forget gate matrices as well as on bias vectors of each gate. Matrix $W_o \in \mathcal{R}^{m,n}$ has $m \times n$ dimensions. After extending the matrix 1 time, dimensions are increased to $W_{eo} \in \mathcal{R}^{m,2n}$. Now matrix W_{eo} has two times columns as compared to W_o .

The same method is applied to rows of the weight matrix in [Figure 3.3](#) of output gate. We are extending the matrix by rows 1 time of output gate. Same process is applied to input and forget gate matrices as well as on bias vectors of each gate. Matrix $W_o \in \mathcal{R}^{m,n}$ has $m \times n$ dimensions. After extending the matrix 1 time, dimensions are increased to $W_{eo} \in \mathcal{R}^{2m,n}$. Now matrix W_{eo} has two times rows as compared to W_o .

It is necessary to change the hidden layer dimension when you are extending the dimensions of matrices and bias vectors. If you miss to change the dimension of hidden layer, some weights will not have effect on training or will be unused or

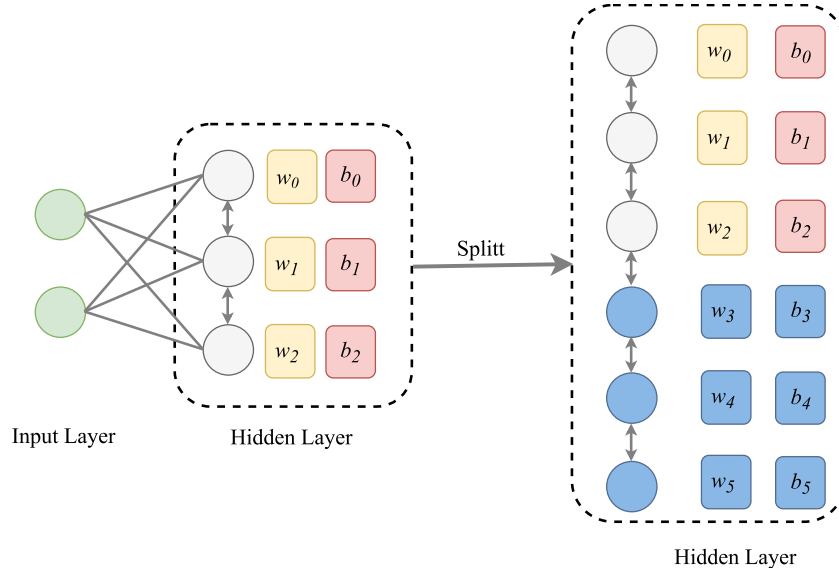


FIGURE 3.1: Parameter Splitting by extending the nodes of hidden layer. $w_{ij} \in W_{ij}$ is weight and $b_i \in b$ is bias value of their respective gates.

$$o_t = \sigma(W_o [h_{t-1}, x_t] + W_{oc}c_t + b_o)$$

$$W_o = \begin{pmatrix} w_{1,1} & w_{1,2} & w_{1,n} \\ w_{2,1} & w_{2,2} & w_{2,n} \\ \vdots & \vdots & \vdots \\ w_{m,1} & w_{m,2} & w_{m,n} \end{pmatrix} \xrightarrow{\text{extend by columns}} W_{eo} = \begin{pmatrix} w_{1,1} & w_{1,2} & w_{1,2n} \\ w_{2,1} & w_{2,2} & w_{2,2n} \\ \vdots & \vdots & \vdots \\ w_{m,1} & w_{m,2} & w_{m,2n} \end{pmatrix}$$

FIGURE 3.2: Extension columns of weight matrix W_o of output layer by 1 time.

have some strange behavior. Same is the case with dimension, you must change the matrices dimension when hidden layer dimensions are changed to have the effect of these changes the model.

Parameters can be modified by increasing the dimensions of matrices for cell gates, weight matrices and bias vectors. Rule of thumb for extending the dimensions of matrix is 1–2 times. Bias vectors are also updated to allow matrix addition rule. Cell size and hidden layer size can be extended to size of weight matrix and bias vector to improve WER. We have performed a various experiments on different configurations of LSTM-RNN and results are shown in [Table 1](#).

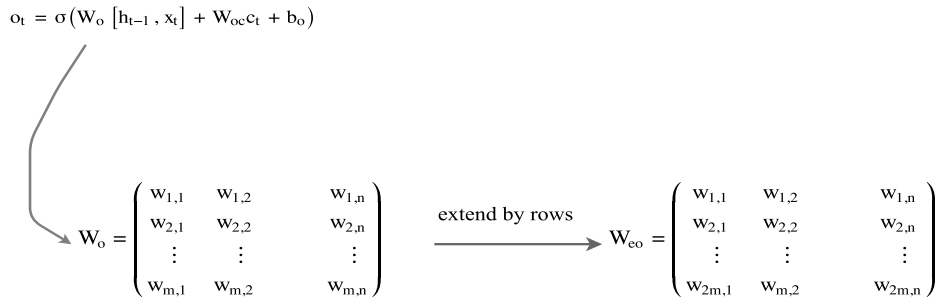


FIGURE 3.3: Extension rows of weight matrix W_o of output layer by 1 time.

3.3 Limit of Splitting

Splitting can effect word error rate and training and cost of LSTM-RNNs. If you apply splitting to decrease the matrix, it will effect negatively and lowers the WER but reduces training time and if you extend too much then training time will increase. So you need to split the model with carefully and according to need of your model for solving the problems.

Well there are some factors which you must considered for applying our discriminative parameter splitting concept are given:

- Size of the data set or number of hours in your data set. If you have a small data set then splitting 1 time is enough and if you have a large data set then splitting can be applied 2–3 times.
- Training time depends on calculation of weights and biases. If you extend more then training time will increase as accuracy to a certain point, after that word error rate may decrease.
- Number of hidden layer in the network. Best configuration can be obtained by experimenting for 3,5 and 7 hidden layer network in LSTM-RNNs. A network with 7 layer is large enough for more than 100 hours of data.

There are other factors which can be used to help out the training such as number of epochs for training, learning rate which helps in accuracy of the model.

Chapter 4

Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are becoming popular for solving sequence problems such as speech recognition, machine translation and natural language processing. RNNs have the ability to remember information for longer period of time due to cycles in their network. These cycles maps the sequences of data (audio signals) with high probability and provides better accuracy as compared to DNN models. RNN use the backpropagation for through time for correcting itself with learning the weights for predicting the data. They have a time frame window for remembering the information and using for longer time interval.

Speech recognition has become an important problem in recent times and gain much popularity as challenging problem. In the past, classic models such as HMM, GMM models were used for recognition of voice and has very limited usage. Due to advancement of hardware and software, machine learning (ML) and artificial intelligence (AI) field grown rapidly for solving such complex problems. A lot of algorithms are available in literature and new models are being proposed for solving such problems. RNNs such as LSTMs have become standard for solving problems based on sequence labeling. We have chosen Peephole LSTM-RNN for our proposed model as it performs better on large data set.

4.1 Peephole LSTM-RNN

There are various variants of RNN such as gated recurrent units (GRUs), LSTM, Peephole LSTM, bidirectional long short term memory (BLSTM) in literature. Each variant is designed for general to specific problems to solve. We have chosen Peephole LSTM-RNN (Gers, Schraudolph, and Schmidhuber, 2002) for our model due to its accuracy on large dataset (Sak, Senior, and Beaufays, 2014). We are using Voxforge data set, which consists of more than 75 hours of data, in our experiments. Peephole LSTM is a variant of LSTM with peephole connections. It uses cell computation as well as hidden layer gates for prediction and classification of data due to ability of remembering information for longer period of time. It has the ability of remembering the information of large signals and sequences which depend on time and provides better accuracy as compared to LSTM-RNN.

4.1.1 Memory Cell

Peephole LSTM-RNN consists of memory cells and each cell has input, output and forget gates. [Figure 4.1](#) shows the LSTM cell with peephole connections. Cell has peephole connections with input, output and forget gates with dotted arrows to their respective gates. Peephole connections provides the information regarding the time using cell state to input, output and forget gate of hidden layer.

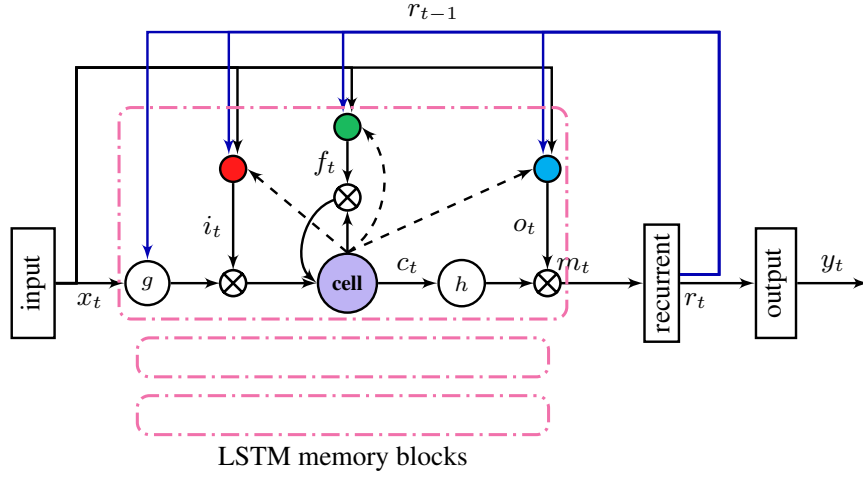


FIGURE 4.1: LSTM-RNN memory cell (Sak, Senior, and Beaufays, 2014). Dotted lines with arrows connecting the cell state to input, output and forget gate are peephole connections. g is a sigmoid function and h is a $\tanh$ function. c_t is cell state and r_t is recurrent connection. i_t , o_t and f_t are input, output and forget gate of hidden layer.

Input, output and forget gates perform sigmoid (σ) operations whereas cell state goes from $\tanh$ operation. Cells in hidden layer are connected through time t .

4.1.2 Equations

A recurrent neural network can be presented by mathematically equations. We are using (Sak, Senior, and Beaufays, 2014) model equations as we have a large data set for experiments. A Peephole LSTM-RNN is given by six equations:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + W_{ic}c_{t-1} + b_i) \quad (4.1)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + W_{oc}c_t + b_o) \quad (4.2)$$

$$f_t = \sigma(W_f[h_{t-1}, x_t] + W_{fc}c_{t-1} + b_f) \quad (4.3)$$

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \sigma(W_c[h_{t-1}, x_t] + b_c) \quad (4.4)$$

$$m_t = o_t \cdot h(c_t) \quad (4.5)$$

$$y_t = \phi(W_y m_t + b_y) \quad (4.6)$$

Here i_t , o_t , f_t are input, output, forget gates, c_t is cell state, m_t is the output of cell state and output gate, and y_t is output of the network. Three activation functions are performed, sigmoid σ activation function is performed on input, output and forget gate. Hyperbolic tangent h or shortly $\tanh$ is performed on cell output and

output gate of hidden layer. Finally softmax ϕ is used on final results for multi-class classification. W_i , W_o , and W_f input, output and forget gates respectively whereas h_{t-1} is the output of previous cell or node. W_{ic} , W_{oc} and W_{fc} are the weights matrices of input, output and forget gate peephole connections. Biases b_i , b_o and b_f are input, output and forget gate vectors whereas b_c is cell state bias vector and b_y is final output layer bias vector.

As we are modifying the Peephole LSTM-RNN architecture for our proposed model, the equations are modified as a variant of Peephole LSTM-RNN architecture.

Following equations represent our proposed Peephole LSTM-RNN as modification of original Equation 4.1, Equation 4.2, Equation 4.3, Equation 4.4, Equation 4.5, Equation 4.6 six equations. We have introduced two new symbols

$$i_t = \sigma(W_{eri}[h_{t-1}, x_t] + W_{eric}c_{t-1} + b_{eri}) \quad (4.7)$$

$$o_t = \sigma(W_{ero}[h_{t-1}, x_t] + W_{eroc}c_t + b_{ero}) \quad (4.8)$$

$$f_t = \sigma(W_{erf}[h_{t-1}, x_t] + W_{erfc}c_{t-1} + b_{erf}) \quad (4.9)$$

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \sigma(W_{erc}[h_{t-1}, x_t] + b_{erc}) \quad (4.10)$$

$$m_t = o_t \cdot h(c_t) \quad (4.11)$$

$$y_t = \phi(W_{ey}m_t + b_{ery}) \quad (4.12)$$

Here e is a splitting parameter variable and r is a matrix or vector which has been randomized by Equation 5.9. Matrices and bias vectors are extended e times and can be extend from 1– n times. For final output layer of LSTM-RNN y_t , softmax is used and represented by ϕ . Detail of these steps are given in section 5.2.

The famous sigmoid activation function is being used for input, output and forget gates in hidden layer. Mathematically, sigmoid is represented as:

$$\sigma(z_t) = \frac{1}{1 + e^{-z_t}} \quad (4.13)$$

here z_t is arbitrary gate for hidden layer input, output and forget gate and produces the output between 0 and 1.

The softmax is used in final output layer to find out the probabilities for multi-class classification for labeling the input with predicted words.

$$\phi(z_t) = \frac{e^{z_t}}{\sum_i e^{z_i}} \quad (4.14)$$

here ϕ which is a softmax and is applied to z_t , the final output layer in Peephole LSTM-RNN.

4.2 Architecture of Peephole LSTM

Peephole LSTM-RNN is a complex RNN network with peephole connections and recurrent connections with each cell or node in hidden layer. The architecture of Peephole LSTM-RNN can be shown in given Figure 4.2.

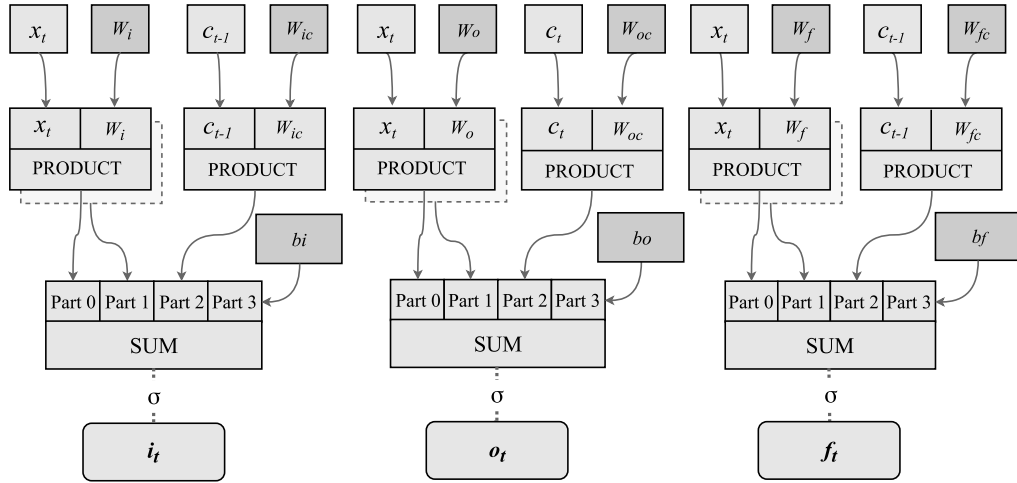


FIGURE 4.2: Input i_t , output o_t and forget f_t gate in Peephole LSTM-RNN. *SUM* and *PRODUCT* are two matrix operations.

We are showing the input, output and forget gate structure in Peephole LSTM. Every gate has its input x_t and weight matrix W such as W_i for input, W_o for output and W_f for forget gate.

After modifying the hidden layer gates and bias vectors and changing the dimension and adding labeling the Figure 4.2 now looks something like this.

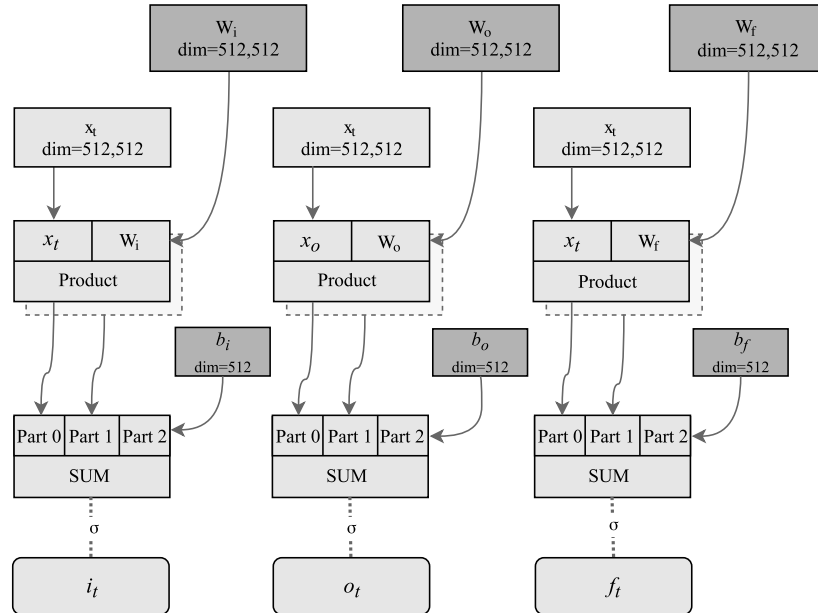


FIGURE 4.3: Input i_t , output o_t and forget f_t gate in Peephole LSTM-RNN with given dimensions of gate matrices and bias vectors with 512 dimensions.

For 2048 matrix dimension, we have:

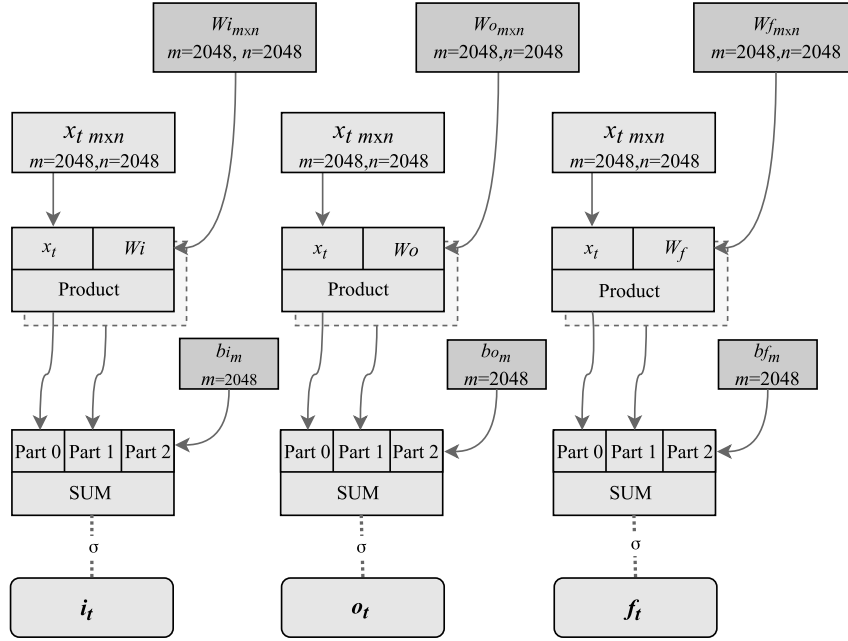


FIGURE 4.4: Input i_t , output o_t and forget f_t gate in Peephole LSTM-RNN with given dimensions of gate matrices and bias vectors with 2048 dimensions

We have applied the 512, 1024 and 2048 splitting on three, five and seven layer LSTM-RNNs are calculated the word error rate and cost value.

Chapter 5

Implementation

5.1 Dataset and Toolkit

We have decided to use the toolkit and dataset which is easily available and can be used freely.

5.1.1 Dataset

We are using Voxforge¹ data set which is freely available for speech recognition purpose. It consists of more than 75 hours of data which is more than enough to build a large acoustic model.

5.1.2 Toolkit

There are many speech recognition toolkits available for speech recognition and general purpose kits for machine learning and artificial intelligence. A comparison is given in (Gaida et al., 2014). We have chosen Kaldi (Povey et al., 2011) for our proposed model as it has DNN as well as RNN models for developing the automatic speech recognition systems.

5.2 Procedure to Train Proposed Model

To build our proposed model of Peephole LSTM, we have divided the procedure into two steps:

- **Build Initial Model:** Preprocess the data and build vocabulary, create a language model of order 3, extract MFCC features and build initial and final model for applying our procedure.
- **Apply splitting procedure** on initial and final acoustic model and perform experiments. Update the weight matrices and bias vectors of hidden layer as well as peephole connections. Apply different LSTM configurations to find out the best configuration of the model. Python scripts are written to modify and update the matrices and vectors of hidden layers. We have applied our procedure on three, five and seven layer LSTM models.

5.3 Build Initial and Final Model

Initial and final models are important and are used repeatedly in our experiments. This process is divided into sub steps given as follows

¹<http://www.voxforge.org/>

5.3.1 MFCC Feature Extraction and Alignment Model

One of the most important feature of speech recognition system is MFCC features which are used build HMM/GMM, DNN and RNN acoustic models. We have build an ARPA language model of order 3 to better provide the probabilities to prediction. Then MFCC features are extracted using the default script given in Voxforge recipe².

Pronunciation Dictionary also knows as lexicon is build from CMU³. Our alignment model is has 2000 HMM state and 11000 GMM with an average of 5 GMM for single HMM state. Alignment model is build by applying double delta+delta-delta method as a supplement to MFCC features. Then we have applied LDA and MLLT to our alignment model for reduction of feature space and speaker independent transformation. Then speaker adaptive training (SAT) and feature space maximum likelihood linear transform (FMLLR) to our model to build a triphone model. We have build our model using this triphone alignment model.

5.3.2 Initialization of Hidden Layer Weight Matrices

There are many proposed initialization algorithms (Fernández-Redondo and Hernandez-Espinoza, 2000; Masood, Doja, and Chandra, 2015) are available. Each has it's own advantages and disadvantages. But we are using an initialization method which is better than most of the model are performs better during training of the neural network in forward and back propagations. We are using (Glorot and Bengio, 2010) initialization method for weight matrices. As we have divided the training of LSTM with non-discriminative and discriminative training. We are using this technique (Glorot and Bengio, 2010) for non-discriminative training for comparison purpose.

$$W_{ij} = U \left[-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}} \right] \quad (5.1)$$

n is the number of input provided to previous hidden layer and the interval $-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}}$ is uniformly distributed with U . $W_{ij} \in \mathcal{R}^D$ is a weight matrix of respective gate in hidden layer. Bias vectors are initialized with normal distribution with $\mu = 0$ and $\sigma = 1$

5.3.3 Creating Initial and Final Acoustic Models

Initial and final acoustic models which are build using alignment model generated subsection 5.3.1. Then we apply linear discriminative analysis (LDA) to build LDA matrix. We then used this LDA feature matrix to create initial and final acoustic model. The recipe provided by Kaldi is used to build these models using the configurations provided in the recipe.⁴

We have created initial and final acoustic model for 3, 5 and 7 layer Peephole LSTM-RNN. We have described the process of modifying the models in section 5.4 to apply our proposed model.

²<https://github.com/kaldi-asr/kaldi/tree/master/egs/voxforge/s5>

³<http://svn.code.sf.net/p/cmuspinx/code/trunk/cmudict>

⁴<https://github.com/kaldi-asr/kaldi/tree/master/egs/voxforge/s5>

5.4 Training and Model Generation

We are using the initial and final model of weight matrix with dimension of 512 columns. We gradually increment the matrix dimension from 512 to 1024 and calculate the results and then increment this model from 1024 to 2048 for experiments. We have repeated this process for three, five and seven layer LSTM-RNN model.

We have applied this process to three, five and seven hidden layer LSTM-RNN. We have divided the modification process in two subsections [subsection 5.4.1](#)

5.4.1 Updation of Initial Acoustic Model

The model generated in [subsection 5.3.3](#) is used as a seeding model in LSTM-RNN. This seed model or initial model generated by Voxforge⁵ has a single hidden layer weight matrices and bias vectors. We have used a complex regular expression to extract and modify the weights and bias vectors. It has five bias vectors b_i, b_o, b_f, b_c, b_y and five hidden layer weight matrices W_i, W_o, W_f, W_y and three hidden layer cell state matrices W_{ic}, W_{oc} and W_{fc} .

The model is by default in binary format, Kaldi provides the utility scripts for converting the binary model to text model and vice versa. We have used ⁶ script to convert the model to text format then we applied our [Algorithm 1](#) to extract the matrices and bias vectors for modification.

Our [Fig. 5.1](#) reads and updates the model and equations [Equation 4.1](#), [Equation 4.2](#), [Equation 4.3](#), [Equation 4.4](#), [Equation 4.6](#). Matrices W_i, W_o, W_f , and bias vectors b_i, b_o, b_f and cell matrices W_{ic}, W_{oc}, W_{fc} are updated by our algorithm.

After modifying the acoustic model it is converted back to binary format so that it can be used in training. Same [6](#) is used with binary flag `binary = false` to convert text format to binary format.

5.4.2 Updation of Final Acoustic Model

Modification of final model is complex and has all the matrices and bias vectors of LSTM hidden layers. We have extracted the three, five and seven layered acoustic model matrices and biases using a complex regular expression from text model which has been converted from binary format.

The formula for calculating the matrices and biases and cell matrices is given as: To calculate the total number of weight matrices in LSTM:

$$m_w = n \times 5 \quad (5.2)$$

To calculate the total number of bias vectors in LSTM:

$$b_w = n \times 5 \quad (5.3)$$

To calculate the total number of cell matrices or peephole connections in LSTM:

$$c_w = n \times 3 \quad (5.4)$$

Here n is total number of hidden layers in LSTM.

Three layered LSTM has 15 weight matrices, 15 bias vectors and 9 cell matrices or peephole connections to input, output and forget matrices. Five layered LSTM

⁵<http://www.voxforge.org/>

⁶http://kaldi-asr.org/doc/nnet3-am-copy_8cc.html

has 25 weight matrices, 25 bias vectors and 15 cell matrices or peephole connections to input, output and forget matrices. Seven layered LSTM has 35 weight matrices, 35 bias vectors and 21 cell matrices or peephole connections to input, output and forget matrices.

These extracted parameters are passed to Fig. 5.4 and Fig. 5.5 and random noise added to these parameters using Equation 5.7.

5.4.3 Algorithm

For modification of initial and final model weight matrices and bias vectors, we have developed an algorithm which we are using for three and five layered LSTM.

Algorithm 1 Discriminative Parameter Splitting

```

1: function EXTRACT(model, name)
    ▷ find gate and cell matrices or vectors using regular expression from given model and name.
    Name can be gates, bias and cell for specifying the needed matrices.
    extracted_matrices ← find(model, name, regular_expression)
2: return extracted_matrices
3: end function
4: function EXTENDC(matrix)
    ▷ extend the columns of matrix and append to itself such that only column dimension is
    increased. Add  $g(w)$  random noise to matrix weights.
5: temporary_matrix ← matrix
6: for column ∈ temporary_matrix do
7:   matrix ← column
8: end for
9: matrix ← f(matrix)
10: return matrix
11: end function
12: function EXTENDR(matrix)
    ▷ extend the rows of matrix and append to itself such that only matrix row dimension is
    increased. Add  $g(w)$  random noise to matrix weights.
13: temporary_matrix ← matrix
14: for row ∈ temporary_matrix do
15:   matrix ← row
16: end for
17: matrix ← f(matrix)
18: return matrix
19: end function
20: initial_model ← Read(initial_model)
21: new_model ← Read(empty_model)
22: g_matrices ← Extract(initial_model, gates)
23: c_matrices ← Extract(initial_model, cell)
24: b_vectors ← Extract(initial_model, bias)
25: m_matrix ← Extract(initial_model, weight)
26: for matrix ∈ {g_matrices} do
27:   matrix ← extendR(matrix)
28: end for

```

FIGURE 5.1: Algorithm for our splitting parameter model

Here *Extract()* method extracts the matrices and bias vectors by providing the model and bias names. It extracts weight and bias vectors provided as parameters. Two functions are used to extend the rows and columns of matrices whereas bias vectors columns are extend with these method. *ExtendC()* method increases the dimension of matrix and increases to $2n$ if matrix has n columns. *ExtendR()* method do the same thing but it increases the rows of matrix being modified. We are adding the random noise from Equation 5.7 to our model.

5.5 Model Configurations and Modifications of Parameters

This section provides the detail about how to change the LSTM configuration for experiment purpose and how to change the dimension of weight matrix and bias vectors.

5.5.1 Configurations of LSTM Layers

It is necessary to modify the dimension in configuration settings of LSTM before performing the experiments on Voxforge data set. If you are experimenting on three layer LSTM with 512 matrix size then you should change the three hidden layer file the configurations and add 512 as hidden layer nodes or cells as well as cell gate configurations. We did this procedure on three, five and seven layer LSTM for proper results. Here are two files with different configurations of LSTM. After changing the dimension from 1024 to 2048, hidden layer 1 configurations will look like this. Every gate has now 2048 dimension as new dimensions for training of layer 1. Layer 2 and 3 should have these changes in their hidden layer gates.

5.5.2 Columns Extension of Matrix

We are using Equation 5.5 for modifying the and incrementing the dimension of columns. Bias vectors are also extending with this equation. Matrix are copied and appending to the old matrix in such a way that only column dimension of matrix is doubled. We then generate a random noise matrix equal to the extend matrix. Matrix R is added to modified matrix. Mathematically,

$$X_c^{m \times 2n} = [X^{m \times n} X^{m \times n}] \quad (5.5)$$

X is a matrix or vector and after extending the matrix it's columns dimension is increased from n to $2n$ and rows dimensions are unchanged. Graphically this method can be shown by Fig. 5.4

```

1 input-node name=input dim=13
2 component name=l0_fixaffine type=FixedAffineComponent matrix=exp/nnet3/lstm_ld8/configs/lda.mat
3 # Input gate control : Wi* matrices
4 component name=lstm1_w_i-xr type=NaturalGradientAffineComponent input-dim=193 output-dim=1024 max-change=0.75
5 # note : the cell outputs pass through a diagonal matrix
6 component name=lstm1_w_ic type=NaturalGradientPerElementScaleComponent dim=1024 param-mean=0.0 param-stddev=1.0 max-change=0.75
7 # Forget gate control : Wf* matrices
8 component name=lstm1_w_f-xr type=NaturalGradientAffineComponent input-dim=193 output-dim=1024 max-change=0.75
9 # note : the cell outputs pass through a diagonal matrix
10 component name=lstm1_w_fc type=NaturalGradientPerElementScaleComponent dim=1024 param-mean=0.0 param-stddev=1.0 max-change=0.75
11 # Output gate control : Wo* matrices
12 component name=lstm1_w_o-xr type=NaturalGradientAffineComponent input-dim=193 output-dim=1024 max-change=0.75
13 # note : the cell outputs pass through a diagonal matrix
14 component name=lstm1_w_oc type=NaturalGradientPerElementScaleComponent dim=1024 param-mean=0.0 param-stddev=1.0 max-change=0.75
15 # Cell input matrices : Wc* matrices
16 component name=lstm1_w_c-xr type=NaturalGradientAffineComponent input-dim=193 output-dim=1024 max-change=0.75
17 # Defining the non-linearities
18 component name=lstm1_i type=SigmoidComponent dim=1024 self-repair-scale=0.0000100000
19 component name=lstm1_f type=SigmoidComponent dim=1024 self-repair-scale=0.0000100000
20 component name=lstm1_o type=SigmoidComponent dim=1024 self-repair-scale=0.0000100000
21 component name=lstm1_g type=TanhComponent dim=1024 self-repair-scale=0.0000100000
22 component name=lstm1_h type=TanhComponent dim=1024 self-repair-scale=0.0000100000
23 # Defining the cell computations
24 component name=lstm1_c1 type=ElementwiseProductComponent input-dim=2048 output-dim=1024
25 component name=lstm1_c2 type=ElementwiseProductComponent input-dim=2048 output-dim=1024
26 component name=lstm1_m type=ElementwiseProductComponent input-dim=2048 output-dim=1024
27 component name=lstm1_c type=BackpropTruncationComponent dim=1024 clipping-threshold=30.0 zeroing-threshold=15.0 zeroing-interval=20 recurrence-interval=1
28 # projection matrices : Wm and Wo
29 component name=lstm1_w_m type=NaturalGradientAffineComponent input-dim=1024 output-dim=256 max-change=0.75
30 component name=lstm1_r type=BackpropTruncationComponent dim=128 clipping-threshold=30.0 zeroing-threshold=15.0 zeroing-interval=20 recurrence-interval=1
31 component name=final_affine type=NaturalGradientAffineComponent input-dim=256 output-dim=1650 max-change=1.50
32 component name=final_log_softmax type=LogSoftmaxComponent dim=1650
33
34 #Component nodes
35 component-node name=l0_fixaffine component=l0_fixaffine input=Append(Offset(input, -2), Offset(input, -1), input, Offset(input, 1), Offset(input, 2))
36 component-node name=lstm1_c_t component=lstm1_c input=Sum(lstm1_c1_t, lstm1_c2_t)
37 # i_t
38 component-node name=lstm1_i1 component=lstm1_w_i-xr input=Append(l0_fixaffine, IfDefined(Offset(lstm1_r_t, -1)))
39 component-node name=lstm1_i2 component=lstm1_w_ic input=IfDefined(Offset(lstm1_c_t, -1))
40 component-node name=lstm1_i_t component=lstm1_i input=Sum(lstm1_i1, lstm1_i2)
41 # f_t
42 component-node name=lstm1_f1 component=lstm1_w_f-xr input=Append(l0_fixaffine, IfDefined(Offset(lstm1_r_t, -1)))
43 component-node name=lstm1_f2 component=lstm1_w_fc input=IfDefined(Offset(lstm1_c_t, -1))
44 component-node name=lstm1_f_t component=lstm1_f input=Sum(lstm1_f1, lstm1_f2)
45 # o_t
46 component-node name=lstm1_o1 component=lstm1_w_o-xr input=Append(l0_fixaffine, IfDefined(Offset(lstm1_r_t, -1)))
47 component-node name=lstm1_o2 component=lstm1_w_oc input=lstm1_c_t
48 component-node name=lstm1_o_t component=lstm1_o input=Sum(lstm1_o1, lstm1_o2)
49 # h_t
50 component-node name=lstm1_h_t component=lstm1_h input=lstm1_c_t
51 # g_t
52 component-node name=lstm1_g1 component=lstm1_w_c-xr input=Append(l0_fixaffine, IfDefined(Offset(lstm1_r_t, -1)))
53 component-node name=lstm1_g_t component=lstm1_g input=lstm1_g1
54 # parts of c_t
55 component-node name=lstm1_c1_t component=lstm1_c1 input=Append(lstm1_f_t, IfDefined(Offset(lstm1_c_t, -1)))
56 component-node name=lstm1_c2_t component=lstm1_c2 input=Append(lstm1_i_t, lstm1_g_t)

```

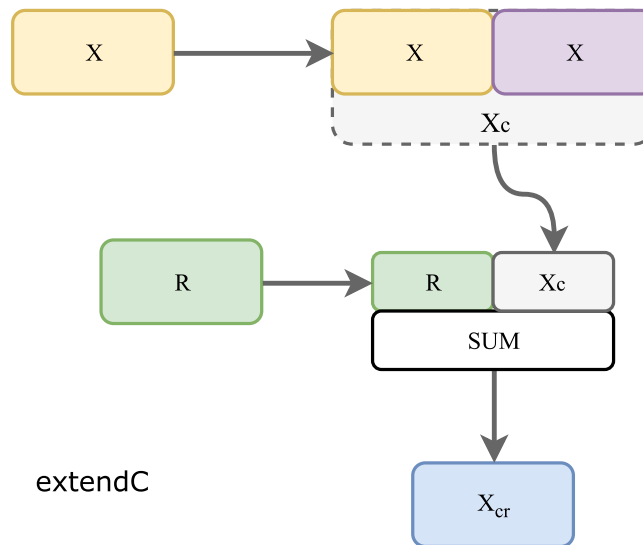
FIGURE 5.2: Configurations of LSTM with 1024 matrix dimension

```

1 input-node name=input dim=13
2 component name=l0_fixaffine type=FixedAffineComponent matrix=exp/nnet3/lstm_ld5/configs/lda.mat
3 # Input gate control : Wix matrices
4 component name=lstm1_w_i-xr type=NaturalGradientAffineComponent input-dim=193 output-dim=2048 max-change=0.75
5 # note : the cell outputs pass through a diagonal matrix
6 component name=lstm1_w_ic type=NaturalGradientPerElementScaleComponent dim=2048 param-mean=0.0 param-stddev=1.0 max-change=0.75
7 # Forget gate control : Wfx matrices
8 component name=lstm1_w_f-xr type=NaturalGradientAffineComponent input-dim=193 output-dim=2048 max-change=0.75
9 # note : the cell outputs pass through a diagonal matrix
10 component name=lstm1_w_fc type=NaturalGradientPerElementScaleComponent dim=2048 param-mean=0.0 param-stddev=1.0 max-change=0.75
11 # Output gate control : Wox matrices
12 component name=lstm1_w_o-xr type=NaturalGradientAffineComponent input-dim=193 output-dim=2048 max-change=0.75
13 # note : the cell outputs pass through a diagonal matrix
14 component name=lstm1_w_oc type=NaturalGradientPerElementScaleComponent dim=2048 param-mean=0.0 param-stddev=1.0 max-change=0.75
15 # Cell input matrices : Wcx matrices
16 component name=lstm1_w_c-xr type=NaturalGradientAffineComponent input-dim=193 output-dim=2048 max-change=0.75
17 # Defining the non-linearities
18 component name=lstm1_i type=SigmodComponent dim=2048 self-repair-scale=0.0000100000
19 component name=lstm1_f type=SigmodComponent dim=2048 self-repair-scale=0.0000100000
20 component name=lstm1_o type=SigmodComponent dim=2048 self-repair-scale=0.0000100000
21 component name=lstm1_g type=TanhComponent dim=2048 self-repair-scale=0.0000100000
22 component name=lstm1_h type=TanhComponent dim=2048 self-repair-scale=0.0000100000
23 # Defining the cell computations
24 component name=lstm1_c1 type=ElementwiseProductComponent input-dim=4096 output-dim=2048
25 component name=lstm1_c2 type=ElementwiseProductComponent input-dim=4096 output-dim=2048
26 component name=lstm1_m type=ElementwiseProductComponent input-dim=4096 output-dim=2048
27 component name=lstm1_c type=BackpropTruncationComponent dim=2048 clipping-threshold=30.0 zeroing-threshold=15.0 zeroing-interval=20 recurrence-interval=1
28 # projection matrices : Wmm and Wpm
29 component name=lstm1_w_m type=NaturalGradientAffineComponent input-dim=2048 output-dim=256 max-change=0.75
30 component name=lstm1_r type=BackpropTruncationComponent dim=128 clipping-threshold=30.0 zeroing-threshold=15.0 zeroing-interval=20 recurrence-interval=1
31 component name=lstm1_affine type=NaturalGradientAffineComponent input-dim=256 output-dim=1664 max-change=1.50
32 component name=lstm1_log_softmax type=LogSoftmaxComponent dim=1664
33
34 #Component nodes
35 component-node name=l0_fixaffine component=l0_fixaffine input=Append(Offset(input, -2), Offset(input, -1), input, Offset(input, 1), Offset(input, 2))
36 component-node name=lstm1_c_t component=lstm1_c input=Sum(lstm1_c1_t, lstm1_c2_t)
37 # i_t
38 component-node name=lstm1_i1 component=lstm1_w_i-xr input=Append(l0_fixaffine, IfDefined(Offset(lstm1_r_t, -1)))
39 component-node name=lstm1_i2 component=lstm1_w_ic input=IfDefined(Offset(lstm1_c_t, -1))
40 component-node name=lstm1_i_t component=lstm1_i input=Sum(lstm1_i1, lstm1_i2)
41 # f_t
42 component-node name=lstm1_f1 component=lstm1_w_f-xr input=Append(l0_fixaffine, IfDefined(Offset(lstm1_r_t, -1)))
43 component-node name=lstm1_f2 component=lstm1_w_fc input=IfDefined(Offset(lstm1_c_t, -1))
44 component-node name=lstm1_f_t component=lstm1_f input=Sum(lstm1_f1, lstm1_f2)
45 # o_t
46 component-node name=lstm1_o1 component=lstm1_w_o-xr input=Append(l0_fixaffine, IfDefined(Offset(lstm1_r_t, -1)))
47 component-node name=lstm1_o2 component=lstm1_w_oc input=lstm1_c_t
48 component-node name=lstm1_o_t component=lstm1_o input=Sum(lstm1_o1, lstm1_o2)
49 # h_t
50 component-node name=lstm1_h_t component=lstm1_h input=lstm1_c_t
51 # g_t
52 component-node name=lstm1_g1 component=lstm1_w_c-xr input=Append(l0_fixaffine, IfDefined(Offset(lstm1_r_t, -1)))
53 component-node name=lstm1_g_t component=lstm1_g input=lstm1_g1
54 # parts of c_t
55 component-node name=lstm1_c1_t component=lstm1_c1 input=Append(lstm1_f_t, IfDefined(Offset(lstm1_c_t, -1)))
56 component-node name=lstm1_c2_t component=lstm1_c2 input=Append(lstm1_i_t, lstm1_g_t)

```

FIGURE 5.3: Configurations of LSTM with 2048 matrix dimension

FIGURE 5.4: Extension of columns of a matrix X , c represents the change in columns and r is random noise by adding R matrix.

SUM is a matrix operation which adds two matrix. Bias vectors b_i , b_o and b_f are extended by this equation. Weight matrices W_i , W_o and W_f are also extended with above mention equation.

We have performed this procedure for two different experiments. First, we performed the extension of columns and did not add the random noise to matrices. We call this technique non-discriminative. Second, we add the random noise to extended matrices and performed various experiments, this technique is called discriminative.

5.5.3 Rows Extension of Matrix

Hidden layer gates such as input, output and forget gate matrices are extended row wise. We modify the W_i , W_o and W_f matrices with Fig. 5.5

$$X_r^{2m \times n} = \begin{bmatrix} X^{m \times n} \\ X^{m \times n} \end{bmatrix} \quad (5.6)$$

Rows are extended 1–2 times for our experiments. Graphically it can be shown Fig. 5.5

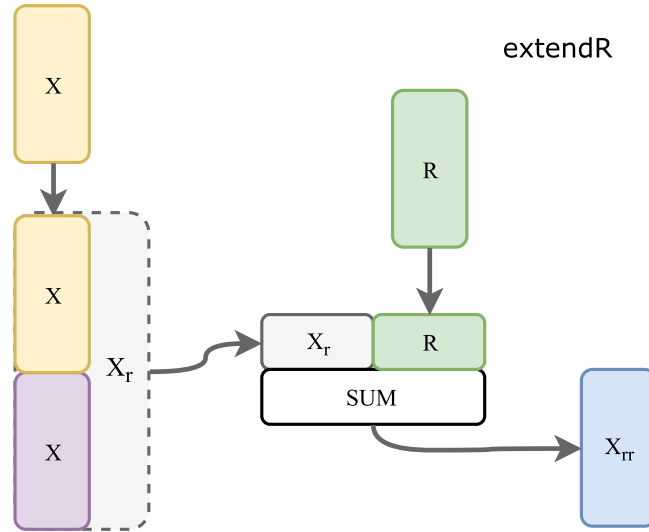


FIGURE 5.5: Extension of rows of a matrix X , r represents the change in rows and r is random noise by adding R matrix.

This method is repeated for non-discriminative and discriminative experiments.

5.5.4 Random Noise

For discriminative splitting, we have used random noise and added this to hidden layer gates and bias vectors. Random noise is generated from normal distribution and constant is multiplied to the generated value to make is very small.

$$f(x) = |(randomize(0,1) - 0.5) \cdot C \cdot \mathcal{N}(\mu, \sigma^2)| \quad (5.7)$$

Here $C = 0.0001$, $\mathcal{N}$ is normal distribution with $\mu = 0$ and $\sigma = 1$.

Depending the sing of the weight matrix, values are updated accordingly using Equation 5.8

$$g(w) = \begin{cases} -f(x) + w & w < 0 \\ f(x) + w & w \geq 0 \end{cases} \quad (5.8)$$

After updating the weight matrix, it will look like Equation 5.9:

$$W_{eri} = \begin{bmatrix} g(w)_{1,1} & \cdots & g(w)_{1,n} \\ \vdots & \ddots & \vdots \\ g(w)_{m,1} & \cdots & g(w)_{m,n} \end{bmatrix} \quad (5.9)$$

Three, five and seven layer LSTM weight matrices are gone through these two modifications and has generates better results as compared to default initialization of weight matrices given in Kaldi (Glorot and Bengio, 2010).

Chapter 6

Results

After successful experiments on our proposed model, results are compiled for three, five and seven layer Peephole LSTM-RNNs. I have performed the proposed technique on Voxforge data set which is shown in Table 6. Few columns has LSTM configurations which is cell dimension, hidden dimension and epochs. Cell dimension is the number of cells in a hidden layer. Hidden dimension is the dimension of Hidden layer. Actually both are the same in such way that if you have same cell dimension and hidden dimension and can be changed according your research. Epoch is the number of training is done for each training phase. In iterations 1–3, we

Iterations	LSTM Layers	Epoch	Cell Dim	Hidden Dim	Parameters (million)	Matrix Size	WER	WER + R	LR
1	3	5	512	512	1	128	16.72	16.62	10^{-3}
2	3	5	512	512	2.8	256	12.60	12.45	10^{-3}
3	3	5	512	512	2.8	512	8.26	7.59	10^{-3}
4	3	10	1024	1024	5.1	512	8.26	7.59	10^{-3}
5	5	5	1024	1024	4.8	1024	7.72	6.56	10^{-3}
6	5	10	1024	1024	9	1024	6.9	6.25	10^{-3}
7	3	10	512	512	2.8	512	7.5	6.92	10^{-3}
8	3	10	1024	1024	5.1	1024	7.26	6.52	10^{-3}
9	3	10	2048	2048	10	2048	7.04	5.69	10^{-3}
*	3	10	4096	4096	19.4	4096	6.87	5.5	10^{-3}
10	5	10	512	512	4.8	512	8.13	7.26	10^{-3}
11	5	10	1024	1024	9	1024	6.90	6.25	10^{-3}
12	5	10	2048	2048	17.5	2048	7.01	5.73	10^{-3}
*	5	10	4096	4096	34	4096	7.76	6.23	10^{-3}
13	7	10	512	512	6.5	512	9.23	9.01	10^{-3}
14	7	10	1024	1024	13.2	1024	8.5	7.92	10^{-3}
15	7	10	2048	2048	26	2048	7.9	6.52	10^{-3}
*	7	10	4096	4096	52	4096	6.69	5.92	10^{-3}

TABLE 6.1: Experiments on three, five and seven layer LSTM

found out that changing the dimension from 512 to 256 and then 128 decreases the WER from 8.26 to 16.72. Experiments 4–6 is used to find out the best configuration for LSTM.

Iterations 7–9 is for three layer LSTM with 512, 1024 and 2048 matrix size with non-discriminative and discriminative splitting word error rate. We have found out that the default (Glorot and Bengio, 2010) produces good results but our method gives lower WER for 512, 1024 and 2048 models.

Iterations 10–12 is for five layer LSTM for 512, 1024 and 2048 hidden dimensions and cell dimension with weight matrix. It can be seen from Table 6 that five layer LSTM shows the best results on 2048 dimensions. Iterations with * consists of very large network and took 2 days for 5 layers to compute the WER and 2.5 days for 7 layer. We have also ran our experiments on seven layer LSTM model to see the effects of our proposed model. Our method still outperforms as compared to default weight matrix initialization.

6.0.1 Cost

Cost of training is important for any RNN or DNN model which uses cross entropy as loss function. It shows that the model is generates less error while predicting the output and finds out the local minimum easily.

We have calculated the cost for three and five layer LSTM-RNN. We have observed that cost decreases smoothly with gradually increasing the training iterations.

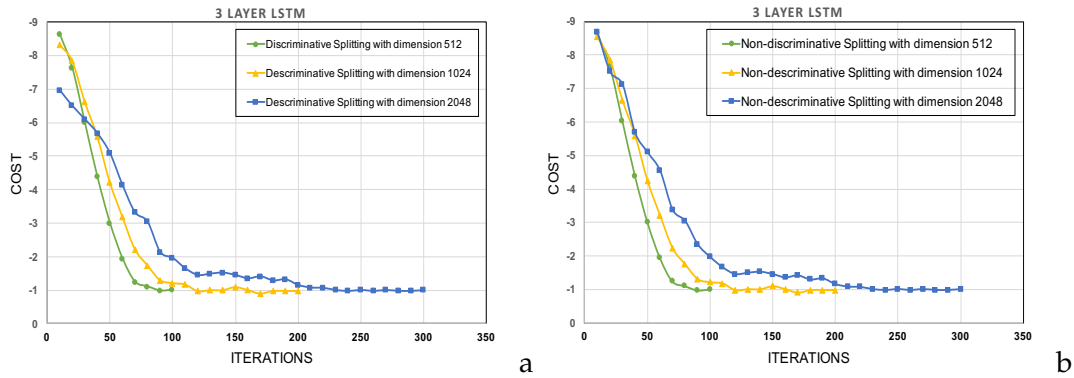


FIGURE 6.1: Loss function comparison of three layer LSTM.

Fig. 6.1 shows the comparison of three layer models. As can be seen from Fig. 6.1(a) that the cost value is -8.84 and gradually decreases to -1 for 512 cell dimension. For 1024 dimension cost is -8.4 and decreases with training iterations and is -1.01 at the 167. We have lower cost value for 2048 dimensions which starts at -6.98 and decreases to -0.9854 on 300 training iterations. Same trend can be seen in Fig. 6.1(b) which shows that extending the dimensions with our proposed model perform better on cost value.

The same behavior can be seen for 5 layer LSTM in Fig. 6.2(a) and Fig. 6.2(b)

6.0.2 WER

Word error rate is important for a speech recognition system which determines how good a system can perform on speech tasks. We have performed two experiments for our proposed model. First, we have calculated the results for non-discriminative splitting and got word error rate for 512, 1024 and 2048 dimensions. Second, we have calculated the results for discriminative splitting for 512, 1024 and 2048 hidden dimensions as can be seen in Fig. 6.3

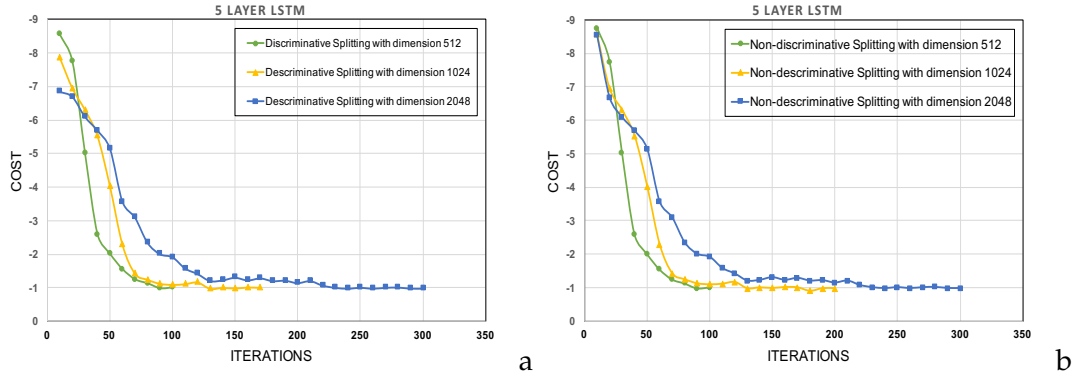


FIGURE 6.2: Loss function comparison of five layer LSTM.

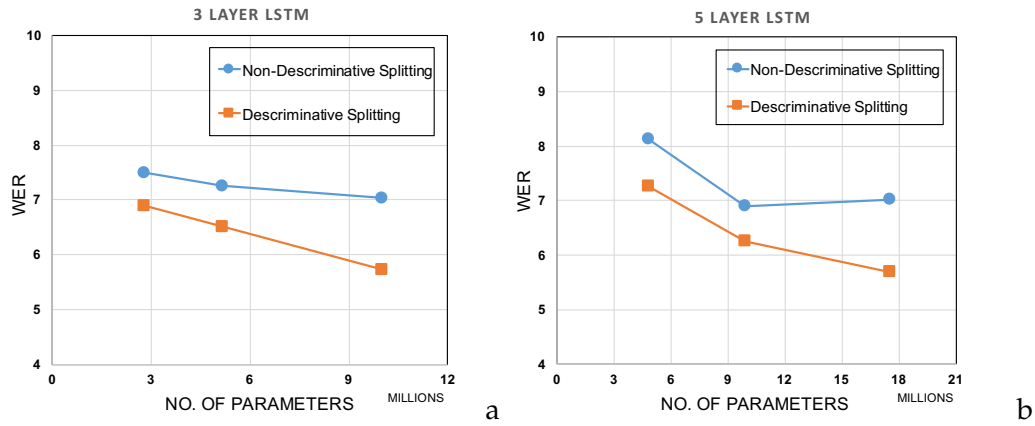


FIGURE 6.3: Effects on WER on Non-discriminative vs discriminative for three Layer (a) and five layer (b)

Fig. 6.3 shows the comparison of three and five layer LSTM. Fig. 6.3(a) shows the word error rate against number of parameters in hidden layers. More parameters give better word error rate as can be seen for 512, 1024 and 2048 hidden dimensions. Same is the behavior for five layer LSTM Fig. 6.3(b).

Chapter 7

Conclusion & Future Work

7.1 Conclusion

Parameter splitting has shown some advantages such as better word error rate, cost of training for Peephole LSTM-RNN. We have applied this model on RNN. Although (Glorot and Bengio, 2010) initialization method shows better results but with our proposed model it even perform better on Voxforge data set. We have boosted the default initialization method by taking the advantages of normal distribution randomness and multiplying a very small constant for positive and negative values of default method. It's a general model and can be used with with DNN and RNN models.

7.2 Future Work

As we have seen that our model performs better on Voxforge data set and give 1.2% improved word error rate with decreasing cost on iterations. We will experiment our model on DNN such as TDNN and RNN such as BLSTM and other DNN and RNN models.

Appendix A

Frequently Asked Questions

A.1 Which toolkit and data set is used for experiments?

We have used **Kaldi** toolkit and **Voxforge** data set which is freely available.

A.2 Is splitting a general model or can be used only in Peep-hole LSTM?

Yes, it is a general model and can also be used for DNN and any RNN model which used weights for accuracy and word error rate.

Bibliography

- Brown, Peter F et al. (1992). "Class-based n-gram models of natural language". In: *Computational linguistics* 18.4, pp. 467–479.
- Fernández-Redondo, Mercedes and Carlos Hernandez-Espinosa (2000). "A comparison among weight initialization methods for multilayer feedforward networks". In: *Neural Networks, 2000. IJCNN 2000, Proceedings of the IEEE-INNS-ENNS International Joint Conference on*. Vol. 4. IEEE, pp. 543–548.
- Gaida, Christian et al. (2014). "Comparing open-source speech recognition toolkits". In: *Tech. Rep., DHBW Stuttgart*.
- Gers, Felix A and Jürgen Schmidhuber (2000). "Recurrent nets that time and count". In: *Neural Networks, 2000. IJCNN 2000, Proceedings of the IEEE-INNS-ENNS International Joint Conference on*. Vol. 3. IEEE, pp. 189–194.
- Gers, Felix A, Nicol N Schraudolph, and Jürgen Schmidhuber (2002). "Learning precise timing with LSTM recurrent networks". In: *Journal of machine learning research* 3. Aug, pp. 115–143.
- Glorot, Xavier and Yoshua Bengio (2010). "Understanding the difficulty of training deep feedforward neural networks". In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, pp. 249–256.
- Masood, Sarfaraz, MN Doja, and Pravin Chandra (2015). "Analysis of weight initialization techniques for Gradient Descent algorithm". In: *India Conference (INDICON), 2015 Annual IEEE*. IEEE, pp. 1–5.
- Murru, Nadir and Rosaria Rossini (2016). "A Bayesian approach for initialization of weights in backpropagation neural net with application to character recognition". In: *Neurocomputing* 193, pp. 92–105.
- Neelakantan, Arvind et al. (2015). "Adding gradient noise improves learning for very deep networks". In: *arXiv preprint arXiv:1511.06807*.
- Povey, Daniel et al. (2011). "The Kaldi speech recognition toolkit". In: *IEEE 2011 workshop on automatic speech recognition and understanding*. EPFL-CONF-192584. IEEE Signal Processing Society.
- Qiao, Junfei, Sanyi Li, and Wenjing Li (2016). "Mutual information based weight initialization method for sigmoidal feedforward neural networks". In: *Neurocomputing* 207, pp. 676–683.
- Sak, Haşim, Andrew Senior, and Françoise Beaufays (2014). "Long short-term memory recurrent neural network architectures for large scale acoustic modeling". In: *Fifteenth Annual Conference of the International Speech Communication Association*.
- Salimans, Tim and Diederik P Kingma (2016). "Weight normalization: A simple reparameterization to accelerate training of deep neural networks". In: *Advances in Neural Information Processing Systems*, pp. 901–909.
- Sundermeyer, Martin et al. (2013). "Comparison of feedforward and recurrent neural network language models". In: *Acoustics, Speech and Signal Processing (ICASSP), 2013 IEEE International Conference on*. IEEE, pp. 8430–8434.

- Tahir, Muhammad Ali, Ralf Schlüter, and Hermann Ney (2011). "Discriminative splitting of gaussian/log-linear mixture hmms for speech recognition". In: *Automatic Speech Recognition and Understanding (ASRU), 2011 IEEE Workshop on*. IEEE, pp. 7–11.